

# Solution for task ‘candies’

## 1 Subtask 1

Perform a naive simulation of the operations.

## 2 Subtask 2

Use prefix sums to determine the number of candies in each box as if the capacity was infinite. Take the minimum of this value and the capacity of the box.

## 3 Subtask 3

Use a lazy propagation segment tree to maintain the minimum and maximum for each range. However, we need to make sure that negative values are reset to zero. For a given segment tree node, let  $a$  represent the minimum value of that node, and  $b$  represent the maximum value of that node.

- Case 1:  $a \geq 0$ . In this case, all values in that node are already correct. Return immediately.
- Case 2:  $b < 0$ . In this case, we update the value of this node by  $-b$ , and proceed to case 3.
- Case 3:  $a < 0, b \geq 0$ . Recursively update the children.

A similar procedure can be done to ensure that the total number of candies stays below the capacity.

It appears that this procedure can take very long to return. Indeed, if all the values are different, this procedure can take up to  $O(n \log n)$  time for a single update. However, the amortized runtime is  $O(\log^2 n)$  per update. The amortized runtime can be proven by keeping track of the number of boxes which have a different number of candies from its neighbor.

## 4 Subtask 4

The key idea behind this subtask can be summarized as follows:

- Find the last point in time in which the box was empty or full.

Let's begin with some observations. Define  $p[i] = v[0] + v[1] + \dots + v[i]$ . If

$$\max(p[i], p[i+1], \dots, p[q-1]) - \min(p[i], p[i+1], \dots, p[q-1]) \geq c[j] \quad (1)$$

Then there exists a point in time after time  $i$  where box  $j$  was either full or empty. For each box  $j$ , we can binary search for the largest  $i$  in which (1) holds. From here, we can calculate the last point in time in which the box was empty or full.

## 5 Full solution

The full solution is very similar to that of subtask 4.

The main difference is that the values of  $p[i]$  are no longer the same for every box. To deal with this, we use a lazy propagation segment tree to update the values of  $p[i]$  as the boxes change. This reduces the time complexity to  $O(n \log^2 q + q \log q)$ . By doing the binary search on the segment tree directly, it can be reduced to  $O(n \log q + q \log q)$ , but this is not required for the full solution.

# Solution for task ‘keys‘

## 1 Subtask 1

Look for connected components.

## 2 Subtask 2

We want to compute  $p[i]$  for each  $i$ . A simple way is to use BFS and restart the BFS procedure whenever we pick up a new key.

## 3 Subtask 3

Again, we want to compute  $p[i]$  for each  $i$ . For each key type  $k$ , store a ‘blocked‘ list of vertices that we are currently unable to reach, but we can reach if we collect key type  $k$ . Whenever we visit a new vertex  $u$ , we classify the edges  $(u, v, c)$  into 3 cases:

- Case 1:  $v$  has already been visited. In this case we do nothing.
- Case 2:  $v$  is not visited and we have the connecting key  $c$ . In this case we simply put  $v$  into the BFS queue.
- Case 3:  $v$  is not visited and we do not have the connecting key  $c$ . In this case, we put  $v$  into the ‘blocked‘ list.

Whenever we collect a new key type, simply clear the ‘blocked‘ list and put the relevant vertices into the queue.

This allows the BFS to run in  $O(n + m + k)$  for every root node. Thus we can compute  $p[i]$  for all  $i$ .

## 4 Subtask 4

Let  $S_u$  represent the set of key types a player starting at  $u$  can get. Suppose that there is a connector joining rooms  $u$  and  $v$  unlocked by key type  $c$ . We can then conclude the following:

- If  $c \in S_u$ , then  $S_v \subseteq S_u$  (\*)

This suggests the following:

Initially,  $S_i = \{r[i]\}$  for all  $i$ . We start with a list of connectors which do not satisfy (\*). If this list is nonempty, we take any connector and apply the operation  $S_u \leftarrow S_u \cup S_v$  where necessary. Once this happens, all edges adjacent to  $u$  are added to the list.

It may be the case where after the list is empty,  $S_u$  may not be correct. For example, consider the input:

- 3 rooms with starting keys  $r[0] = 0, r[1] = 1, r[2] = 2$
- 2 connectors, connecting rooms (0 and 1), and (1 and 2). Both connectors require key type 0 to unlock.

A player starting at room 0 is able to collect all 3 types of keys, however the algorithm will stop at  $S_0 = \{0, 1\}, S_1 = \{1\}, S_2 = \{2\}$ .

We put this issue aside temporarily and pretend that we have correctly calculated  $S_u$  for all  $u$ , and explain later why this does not actually matter for the purposes of solving this problem.

If there is a connector  $(u, v, c)$  and  $c \in S_u$ , then we may draw an edge  $u \rightarrow v$ . After identifying the strongly connected components, we simply need to calculate  $p[i]$  for the tail components.

Back to the situation where  $S_u$  may not be correct. Observe that every strongly connected component must have the same set  $S_u$  for each  $u$  in the component. This is because an edge  $u \rightarrow v$  is drawn only when  $S_v \subseteq S_u$ . If a player  $u$  obtains a key not in  $S_u$ , then this key must be obtained in some  $v$  which is in a different component from  $S_u$ . However, when this happens, it means that the component containing  $u$  is not a tail component. Since we only want the minimal  $p[i]$ , this does not matter.

Now let's analyze the time complexity of this algorithm. Observe that each edge is added to the list at most 60 times since an edge is only added when  $S_u$  is changed. The complexity of this operation is therefore  $O(m \cdot \max c[i])$  (assuming that the set union operation takes  $O(1)$  time, which is acceptable since we only need to store 30 bits).

## 5 Full solution

First, assign every room a root. Initially, the root of every room is itself. A round consists of the following:

- Identify all the rooms which root is itself
- For each of this rooms, run the BFS algorithm described in subtask 3, terminating early if we are able to reach a room with a different root.
- Whenever an early termination occurs, we write down a list of pairs  $(x, y)$  where 'a BFS search starting at  $x$  reaches a room which root is  $y$ '.

- At the end of the round, for all such pairs  $(x, y)$ , every room which root is  $x$  (including  $x$  itself), is now assigned a root of  $y$ . It does not matter how cycles are resolved, as long as each cycle contains a consistent root.

We can use the union find data structure to maintain the roots. The algorithm only needs to be ran for  $\log_2(n)$  rounds. This can be proven by drawing out the merging tree - if in any iteration a room is not involved in any merge, then it will never be involved in any future merges.

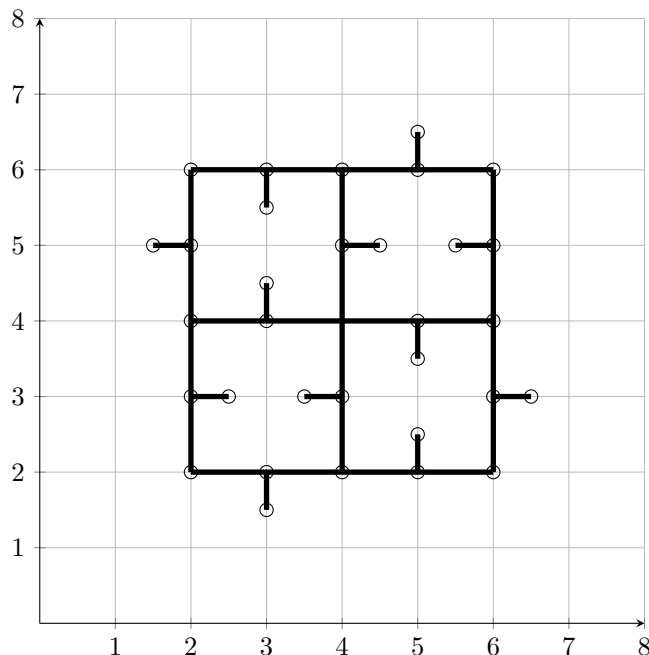
Lastly, we finish by doing BFS on each of the roots.

## Solution for ‘parks’

It turns out that a construction always exists whenever it is possible to connect the fountains by roads. For a given set of roads, methods like bipartite matching or 2-SAT can determine if that set of roads can produce a solution. Therefore, when the set of roads is unique (subtask 4), 2-SAT is sufficient.

A possible approach is to simply construct every road whenever we have a pair of fountains distance 2 apart. This approach works whenever there is no  $2 \times 2$  square (subtask 5).

For the full solution, we tile the plane in a chessboard manner and always assign benches according to the following rule:



Now we sort the fountains by lexicographical order (increasing  $x + y$  also works).

Whenever we encounter a new fountain  $(x, y)$ , we connect this fountain to  $(x, y - 2)$  if the fountain  $(x, y - 2)$  exists and its corresponding bench is not occupied.

We do the same for  $(x - 2, y)$ .

## Solution for task ‘dna’

Let  $m$  denote the length of a query, i.e.  $m = y - x + 1$

A solution exists for a given query if and only if the multiset of characters in  $a[x..y]$  is the same as the multiset of characters in  $b[x..y]$ .

### 1 Subtask 1

Constraint:  $y - x \leq 2$

There are at most 3 characters in the query, therefore at most two swaps will suffice.

Assuming a solution exists, check if there is a one swap solution, if yes, the distance is 1, else the distance is 2.

### 2 Subtask 2

For two characters, we can count the number of ‘A’s that should be ‘T’s and ‘T’s that should be ‘A’s.

Each swap can fix a mismatch pair, therefore a solution is to count the number of mismatch pairs.

This can be done in  $O(m)$  time per query as the number of queries is small.

### 3 Subtask 3

The is similar to subtask 2 except there are more queries.

To speed up the counting of mismatches for each query, we can pre-compute the number of mismatched pairs for all possible prefixes in  $O(n)$  time.

Using the prefix table, we can determine the number of mismatched pairs for any substring in constant time.

The total time complexity will be  $O(n + q)$

### 4 Subtask 4

For three characters, we can first look for any swap that will result in fixing two characters. Looking for and applying these swaps takes  $O(m^2)$  time.

After that, the remaining mismatches will take 2 swaps to fix three characters. Looking for and applying these swaps can also be done in  $O(m^2)$  time.

The total time complexity will be  $O(qm^2)$

## 5 Full solution

We need to pre-compute mismatches for each prefix as we did in subtask 3. This can be done by recording the number of 'P's that should be 'Q's for every P and Q and every prefix using  $n \times 3 \times 3$  mismatch matrices.

Given a query, we can compute the mismatch matrix for that query in constant time. This will allow us to count the number of swaps that fixes two characters in constant time and subtract it from the mismatch matrix.

The remaining mismatches requires 2 swaps to fix three characters and we can also count these in constant time.

The total time complexity will be  $O(n + q)$

## 6 Solving for larger alphabet sizes

We can view the mismatch matrix as a multigraph, where the nodes are the characters in the alphabet and there are edges from  $u$  to  $v$  for each position  $i$  where  $a[i] = u$  and  $b[i] = v$

It is optimal to proceed in the following greedy fashion for graphs with 5 or fewer nodes.

1. find cycles of length 2 and removing them with 1 swap
2. find cycles of length 3 and removing them with 2 swaps
3. find cycles of length 4 and removing them with 3 swaps
4. find cycles of length 5 and removing them with 4 swaps

However, this approach fails in general when the alphabet size 6 or larger.

# Solution for ‘dungeons’

It is stated in the constraints section that the player will eventually reach dungeon  $n$ . Why is this so?

Observe that the player’s size increases after each step. Once the player’s size is greater than  $10^7$ , he wins in every chamber. From there, the chamber number can only increase.

## 1 Subtask 1

Simulate the movement of the player stepwise in a naive way.

## 2 Subtask 2

We make the following observation:

Once the player loses, the size of the player must double. As such, the number of losses is bounded by  $\log_2 S$ , where  $S = \max s[i]$ .

For each dungeon  $i$ , we draw an ‘outgoing edge’  $(s[i], w[i])$  indicating the gain in size and the next location if a player wins.

We may accumulate edges as follows:

$$(s[i], w[i]) \star (s[w[i]], w[w[i]]) = (s[i] + s[w[i]], w[w[i]])$$

to indicate the movement of the player when the player wins twice in a row.

Next we make this observation:

A player can cross that if the player’s size (before the increment) is larger than every  $s[i]$  on the path of this player.

As such, we can record down the largest  $s[i]$  for every edge and build power-of-2 tables.

Using power-of-2 tables, we can binary search till the point which the player loses next.

It takes  $O(n \log n)$  time to build the power-of-2 tables, after which finding the next loss will take  $O(\log n)$  time.

The total time complexity is  $O(n \log n + q \log n \log S)$ .

### 3 Subtask 3

Similar to the previous subtask, we accumulate edges by the following rule:

$$(p[i], l[i]) \star (p[l[i]], l[l[i]]) = (p[i] + p[l[i]], l[l[i]])$$

Again, we build power-of-2 tables.

The movement of the player can be simulated until  $z \geq s$  (unless the player reaches dungeon  $n$  first). After that, we can simulate all the way to the ending chamber using a simple prefix sum.

The movement of the player can be broken down into two phases. The first phase consists of the movement before the player's first win. The second consists of the remaining movements.

The first phase can be solved by using power-of-2 tables.

The second phase can be solved by using a static prefix sum to the winning dungeon. Time complexity  $O((n + q) \log S)$ .

### 4 Subtask 4

Let the distinct values of  $s$  be  $t_0, t_1, \dots, t_4$ .

Consider a player with size  $z$ , where  $t_0 \leq z < t_1$ . We wish to simulate the movements of this player until  $z \geq t_1$  (or  $x = n$ ).

We say that dungeon  $i$  is winning if  $s[i] \leq t_0$ , and losing if  $s[i] \geq t_1$ .

Following the above subtasks, we draw an edge  $(p[i], l[i])$  for losing dungeons and  $(s[i], w[i])$  for winning dungeons.

The edges can be accumulated as before. Construct power-of-2 tables until the player's size exceeds  $t_1$ .

We can repeat the above process until  $x = n$ . Time complexity:  $O(k(n + q) \log S)$  where  $k$  is the number of different sizes.

### 5 Subtask 5

If we use the approach in the previous subtasks, there are too many tables to build. The idea is to group dungeons of similar size together.

We will group the dungeons by the  $\log_2 s[i]$ , and construct phases  $1, 2, \dots, \log_2 10^7$ .

For phase  $k$ , dungeon  $i$  is winning if  $s[i] < 2^k$  and losing otherwise.

Now consider a player with size  $z$ , where  $2^k \leq z < 2^{k+1}$ . We cannot simply jump in the naive way until  $z \geq 2^{k+1}$ . This is because we have declared all dungeons with size  $\geq 2^k$  as winning. Thus the player might win before his size exceeds  $2^{k+1}$ . This contrasts with the previous situation where there cannot be any dungeon with size between  $t_0$  and  $t_1$ .

As such, we have to store an extra parameter in the edge: a 'winning threshold'.

Let the winning threshold be  $s[i]$  for a winning edge and  $\infty$  for a losing edge. Then we can now accumulate edges via the following rule:

```
(inc[i], out[i], threshold[i]) * (inc[out[i]], out[out[i]], threshold[out[i]])
= (inc[i] + inc[out[i]], out[out[i]], min(threshold[i], threshold[out[i]] - inc[i]))
```

A player can now travel past an edge if

1. The player's size (before the increment) is less than the threshold.
2. The player's size (after the increment) is less than  $2^{k+1}$ .

In actual fact, condition (2) can be dropped, as the threshold prevents a player from accidentally losing a dungeon that should be won. Observe that once we run the simulation until (1) is met and win that chamber, the player's size must have increased by  $2^k$ , so that the player's size is at least  $2^{k+1}$ .

Therefore, condition (1) is only triggered at most  $\log S$  times.

By building the power-of-2 tables this way, it takes  $O(\log S)$  time to advance 1 phase. Therefore the total time taken is  $O((n + q) \log^2 S)$ .

## 6 Subtask 6

The solution for subtask 5 will run out of memory as it requires  $O(n \log^2 S)$  memory.

It is possible to reduce the time and memory consumption taken to build the power-of-2 tables to  $O(n)$  for each table. There are a few ways to do it.

One way is to use the technique described in <https://codeforces.com/blog/entry/74847>, which basically speeds up the building of the tables. Another solution is to adjust the phases such that each phase consists of strengths  $[b^k, b^{k+1} - 1]$  for some larger  $b$ . In practice,  $b = 8$  should work.

We will present an alternative solution here.

### 6.1 Alternative solution

For every dungeon  $i$  and every phase  $p$ , we compute the path from dungeon  $i$  to the next dungeon of phase  $p$  (we do not allow the empty edge, even if dungeon  $i$  is of phase  $p$ ). Take note that when computing the path, we use the behavior of phase  $p$  (i.e. win if  $s[i] < 2^p$ , lose otherwise). The size increase, threshold, and outgoing location are tracked as described in the earlier subtasks. We call these edges 'superedges'. Among these superedges, call a superedge special if dungeon  $i$  is of phase  $p$ . This gives rise to  $n \log S$ , which are split into  $\log S$  phases.

We build a power-of-2 table on the special superedges. This uses  $O(n \log S)$  pre-computation time as there are only  $O(n)$  superedges.

To answer the queries. We have a variable  $p$ , denoting the player's phase. Initially,  $p = 0$ . Now suppose that the player is at dungeon  $x$  and has phase  $p$ :

- Case 1:  $s[x] < 2^p$  or  $s[x] \geq 2^{p+1}$ . Look for the superedge of dungeon  $x$  and phase  $p$ . If we can follow that edge, follow it and proceed to case 2. If not, increment  $p$ .

- Case 2:  $2^p \leq s[x] < 2^{p+1}$ . Follow the superedges of phase  $p$  (note that no matter how many steps we take, we only remain in dungeons of phase  $p$ ). Proceed until we cannot cross any more superedges in phase  $p$ . Simulate 1 step using the naive method, and then increase  $p$ .

The reason why this algorithm works depends on the following invariant:

- When the player has phase  $p$ , the player will not lose in any dungeon  $x$  with  $x < 2^p$ .

To see why this is true, we only need to focus on what happens when  $p$  is incremented. (In the beginning,  $p = 0$  so the condition is trivially true.)

Suppose we reach case 1 and we cannot pass through the superedge leading to phase  $p$ . This means that we must have won somewhere along this path of that superedge. What is the size of this dungeon? It must be at least  $2^p$  by assumption. However, along this path there cannot be any dungeons with size between  $2^p$  and  $2^{p+1}$ , because of the way the superedges are constructed. Therefore we must have won a dungeon of size at least  $2^{p+1}$ .

Suppose we reach case 2. This time round, the argument from the previous paragraph mostly holds, except for the fact that we may win on the dungeon the player is currently on. Therefore, we are required to take 1 baby step to simulate.

Each query can be answered in  $O(\log^2 S)$  time.

# Solution for ‘registers’

## 1 Subtask 1

The key is to set up the following procedure:

- Given 3 registers  $r_a, r_b, r_c$ , assign  $r_c = r_a$  if  $r_a = r_b$ , or 0 otherwise. (Here we slightly abuse the notation  $r_a, r_b, r_c$  to refer to either the register itself or the value stored inside).

To do so, take a new register  $r_d$  and set  $r_d = r_a \text{ XOR } r_b$ . Now, if  $r_a = r_b$ ,  $r_d$  stores all 0, else at least one bit of  $r_d$  is 1. We take  $r_d \text{ OR } (r_d \text{ shifted left})$  a total of 4 times, and then take  $r_d \text{ OR } (r_d \text{ shifted right})$  a total of 4 times.

Once we do this,  $r_d$  has its 4 least significant bits all 0 if  $r_a = r_b$ , or all 1 otherwise. One we do this, we can set  $r_c = (\text{NOT } (r_d)) \text{ AND } (r_a)$ .

As there are only 16 possible inputs, we can encode all 16 cases.

## 2 Subtask 2

We start with the operations `and(1,0,k)`, `not(2,1)`, `add(3,0,2)`. After these instructions are executed,  $r[3][k]$  will tell us which of  $r[0]$  and  $r[1]$  is larger.

From here, a similar branching approach will enable us to solve the problem.

## 3 Subtask 3

Proceed as with the earlier subtask. Repeatedly take the minimum between two numbers.

## 4 Subtask 4

Use a tournament structure and parallelize the operations.

## 5 Subtask 5

Use bubble sort idea to repeatedly bubble the smallest element to the bottom.

## 6 Subtask 6

Observe that bubble sort can be parallelized. Essentially, do the following:

Initially our register contains  $a[5], a[4], a[3], a[2], a[1], a[0]$ . Using the ideas from the above subtasks, we can modify the values to

$\max(a[5], a[4]), \min(a[5], a[4]), \max(a[3], a[2]), \min(a[3], a[2]), \max(a[1], a[0]), \min(a[1], a[0])$ .

Afterwards, we reverse the parity (i.e.  $a[4], a[3]$  now becomes  $\max(a[4], a[3]), \min(a[4], a[3])$ ).

Repeat this for  $n$  times.